Bias in Bios: Project 1 Study Guide

Nov 6, 2026 · @lsha

We predict a person's occupation (28 options) from their online bio, compare 9 systems (3 representations × 3 models) with cross-validation and a Bayesian test, and check whether the systems work equally well for women and men. The best system is an MLP on TF-IDF features (macro-F1 0.754), and every system amplifies the gender imbalance of its training data. The sections below follow the six "Get to work!" questions, the ones the TA picks from: for each, what we did, why, and short answers to likely questions.

At a glance

The 30-second story. We read a bio and guess one of 28 occupations. Professor alone is 30% of the bios, so we judge systems by **macro-F1** (every occupation counts equally), not accuracy. We built 9 systems and compared them on 6 cross-validation splits with a Bayesian test. **One hidden layer helps, unless the words are weighted with TF-IDF**: then the simple log-linear model is practically as good. Our own deeper MLP, after a redesign, matches the one-layer MLP but does not beat it. Every system **amplifies gender imbalance**: in occupations dominated by women, women are recognised more often, and vice versa. Deleting pronouns costs no accuracy and shrinks those gaps by about a fifth, but does not remove them.

What	Number to remember
Bios used (train, deduplicated)	257,081 (of 257,478)
Occupations	28; professor 29.8%, rapper 0.35% (84× smaller)
Share of women	46.1% overall; 9.7% (rapper) to 92.9% (dietitian) per occupation
"Always say professor" baseline	accuracy 0.298, macro-F1 0.016
Pronouns reveal the gender label	in 94.9% of bios, with 100% agreement
Tokenizer	own BPE, 10,000 tokens; a bio is about 77 tokens
Cross-validation	3 folds × 2 seeds = 6 scores per system, stratified by occupation × gender
Best system	MLP + TF-IDF: macro-F1 0.754 ± 0.002, accuracy 0.811
Worst system	log-linear + bag-of-words: macro-F1 0.719

What	Number to remember
ROPE	±0.01 macro-F1 (about 3× the typical fold-to-fold noise of 0.0031)
Gender gap vs % women (best system)	correlation r = 0.86
Biggest gaps	model +0.49 (women recognised far more), surgeon −0.21 (men recognised more)
Deleting pronouns	macro-F1 0.754 → 0.751; RMS gender gap 0.149 → 0.116 (−22%)

What the code does

Every system runs the same pipeline: **text → tokens → a vector of numbers → a model → 28 probabilities → the most probable occupation**. The notebook (`notebooks/bias-in-bios-week1.ipynb`) runs it in this order:

1. **Load and explore the data (§1).** Count occupations and genders, look at bios, find duplicates, templates and give-away keywords.
2. **Make the splits (§2).** Remove duplicates, then cut the 257,081 bios into 3 stratified folds, twice (2 seeds), and save the folds.
3. **Train the tokenizer (§3).** Learn 10,000 BPE tokens from the bios and turn every bio into a list of token ids, once.
4. **Build the representations (§4).** One big sparse table of token counts (bios × tokens); R1, R2 and R3 are all made from it.
5. **Define and tune the models (§5).** Three architectures; pick one setting per system on a hold-out from training data.
6. **Define the metrics (§6).** Macro-F1, accuracy, balanced accuracy; recall per gender.
7. **Cross-validate all 9 systems (§7).** Each trains 6 times (2 seeds × 3 folds); results are saved to disk.
8. **Compare (§8–§9).** Tables and plots, then the Bayesian comparison with a ROPE.
9. **Fairness (§10–§11).** Gender gaps per occupation, then retrain the best system with pronouns deleted.

Where the code lives. Notebook cells stay short; longer code is in small files in `src/bib/` . The tutorial's own code (`src/part1/`) is reused where it fits and never changed.

File	What it does	Used in
`bib/tokenizer.py`	trains our BPE tokenizer; encodes every bio into token ids	§3
`bib/encoders.py`	builds the count table; turns it into R1 indicator, R2 bag-of-words, R3 TF-IDF (`fit` on training rows, `transform` both)	§4, §7
`bib/models.py`	the 3 models behind one interface (`fit` , `predict`); the neural training loop with early stopping	§5, §7
`bib/tuning.py`	tries each candidate setting per system and keeps the best	§5
`bib/evaluation.py`	scores (macro-F1, accuracy, balanced accuracy), recall per gender, gaps, confusion matrix	§6, §8, §10
`bib/cv.py`	runs one system on every split and saves predictions and scores (`results/week1/<system>.pkl`)	§7
`bib/scrub.py`	deletes he/she/his/her/him/hers/himself/herself/mr/mrs/ms from a bio	§11
`bib/data.py`	rebuilds the exact pool and folds from `results/week1/splits.json`	`scripts/run_grid.py`
`bib/plots.py` , `bib/style.py`	every chart, in one style (women orange, men blue)	all
`part1/smlp.py` (tutorial)	`SparseMLP` , the one-hidden-layer model, reused as-is	§5
`part1/analysis.py` (tutorial)	`BenavoliCorrelatedT` , the Bayesian comparison, reused as-is	§9, §11
`part1/experiment.py` (tutorial)	its `train_tokenizer` , used only to show the tutorial tokenizer next to ours	§3

What we did *not* reuse from the tutorial, and why: `loglin.fit` and `smlp.train_neural_network_model` build a dense vector per bio (≈ 10 GB for our data) and the second picks the best epoch on the validation fold, which inflates scores; `experiment.cross_validation_split` does not stratify; `experiment.train_tokenizer` is set up for AG News text.

Q1 · The dataset

"Choose a dataset. Spend some time understanding how the dataset looks and some statistics of it."

What it is. `LabHC/bias_in_bios` on Hugging Face: about 400k English online bios collected from the web (Common Crawl) by De-Arteaga et al. (2019); this version is the one used by Ravfogel et al. (2020). A bio was kept if its first sentence read *"[Name] is a(n) [occupation]"* for one of 28 occupations. That occupation became the **label**, and that sentence was then **removed**, so the model must infer the job from the rest (`hard_text`).

Gender (0 = male, 1 = female) was **inferred from pronouns**. It is binary, a proxy rather than self-reported identity, and non-binary people are absent. We never give gender to the model; we use it only to check fairness.

Split	Bios	Use this week
train	257,478	all cross-validation (after removing duplicates: 257,081)
dev	39,642	untouched
test	99,069	untouched

What we found, and what we decided because of it

Finding	Decision
Professor is 29.8% of bios; 6 jobs are under 0.5% each; largest/smallest = 84×	macro-F1 as main score; stratified folds
"Always professor" scores 0.298 accuracy but 0.016 macro-F1	accuracy is misleading here; models must clearly beat 0.30 / 0.04
% women per job ranges from 9.7 (rapper) to 92.9 (dietitian); smallest group is 88 female rappers	fairness analysis per occupation; combine all folds' predictions before measuring it
A pronoun-counting rule recovers the gender label in 94.9% of bios, 100% correctly	"gender is not an input" is only true on paper, so we must measure its effect
382 duplicate copies; 15 texts carry different occupation labels (label noise)	deduplicate before cross-validation
6.8% of bios share a template opening (e.g. a nurse-directory template)	keep them (real data), expect confusions between medical jobs

Finding	Decision
Median bio is 55 words; women and men write equally long bios	motivates a length-normalised representation (TF-IDF)
Some jobs name themselves ("dentist" in 79% of dentist bios, 87% reliable); others share words ("model", "teacher")	predicts which jobs will be easy or hard; checked in the results

Likely questions

- **Why this dataset?** It is a real, large (257k) classification task with a built-in fairness question: does a classifier work equally well for women and men? That question carries into Projects 2–3.
- **Why is accuracy a bad metric here?** Because of imbalance: guessing "professor" every time already gets 30% accuracy while being useless for 27 of 28 jobs.
- **What is `hard_text`, and why does it matter?** The bio without the sentence that names the job. Without it, the task would be trivial (find the job word in sentence one).
- **How reliable are the labels?** Mostly fine, but there is noise: 15 duplicated texts have different labels, templates span several jobs, and some bios describe two jobs (a composer who is also a professor).
- **How reliable is the gender label?** It was made from pronouns, so it is binary and can be wrong for people whose bios use other pronouns or none (5% have no clear pronoun).
- **Do the gender shares reflect the real world?** No. They describe who has an English bio online, not the workforce.
- **Why not use dev and test?** All design choices are made with cross-validation on train, so test stays an honest final check for later projects.

Q2 · The tokenizer

"Do you want to use the tokeniser we have developed? Do you need to train a new one? What hyperparameters? What data? Is your dataset too small?"

What we did. We trained our **own BPE tokenizer** (Hugging Face `tokenizers`) on all 257,081 deduplicated training bios: 10,000 tokens, merges only for pairs seen at least 5 times. It takes about 4 seconds. Every bio is then encoded once into token ids, which all 9 systems reuse.

How BPE works, in four steps (Sennrich et al., 2016):

1. Start with single characters as the vocabulary (plus `[UNK]` for characters never seen).
2. Count every pair of neighbouring units in the corpus.
3. Merge the most frequent pair into a new unit and record the merge (e.g. `n` + `u` → `nu`, later `nur` + `se` → `nurse`).
4. Repeat until the vocabulary has 10,000 units, or no pair occurs at least 5 times.

To tokenize new text, split it into words (the *pre-tokenizer*), then replay the recorded merges in the same order. Frequent words end up as one token, and rare words (names, places, typos) are split into pieces rather than becoming "unknown".

Our choices, and why

Choice	Why	Alternative we rejected
Train our own tokenizer, not the tutorial's	Our text is professional bios, not news; job words (*chiropractic, paralegal*) should be whole tokens	Tutorial's AG News tokenizer: domain shift, job words would be fragmented
Train it on our own bios	257k bios ≈ 20M tokens is plenty; the "too small" worry does not apply	A bigger external corpus: not needed
10,000 tokens	21 of 28 job names become one token, and the feature space stays small (first layer of the MLP = 10,000 × hidden)	1,000 (tutorial): fragments job words; 30k+: more, rarer features and larger models
Minimum frequency 5	stops merges for one-off junk (phone numbers, rare names)	2 (tutorial): more noise in the vocabulary
Lowercase + NFKC Unicode normalisation	*He/he*, *Professor/professor* become one token; frees vocabulary slots	Keep case: wastes slots on duplicates
Split punctuation off before BPE	otherwise *"engineer."* and *"engineer"* are different tokens	Tutorial's plain `Metaspace`: punctuation stays glued to words

What came out. 21 of 28 job names are a single token (split: *chiroprac+tor*, *comed+ian*, *rapper*, and four two-word names). Gender words (*she, he, her, his, husband, wife*) are single tokens. Rare names are split: *Brittni* → brit + t + ni. A bio is a median of 77 tokens; 0% unknown tokens.

Likely questions

- **Is training the tokenizer on all of train leakage?** No labels are involved, so no answer leaks. It does see the characters of bios that later serve as validation folds, but it is unsupervised and identical for all 9 systems, so the comparison stays fair. The strict alternative is to train it on `dev` only.
- **What if our dataset had been too small?** Train on a larger unlabeled corpus of similar text (e.g. other web bios or Wikipedia biographies), then use it on our data.
- **What does a bigger vocabulary change?** Longer, more word-like tokens, which are more informative, but more features, rarer counts per feature and more parameters. A smaller vocabulary means more fragments, which carry less meaning on their own. Bigger is not automatically better.
- **What does a higher minimum frequency do?** Fewer merges for rare pairs, so less junk in the vocabulary, but rare real words stay split.
- **Downside of lowercasing?** We lose case cues, e.g. the ALL-CAPS "NURSE PRACTITIONER" in templates, or proper nouns versus common words.
- **Are BPE tokens morphemes?** No. They are frequent character strings, not meaningful units; *chiroprac+tor* happens to look morphological, *brit+t+ni* does not.
- **Why not character-level?** A single character carries almost no meaning, and a bag of characters loses nearly all information.
- **Why not NLTK word tokenization?** It gives an open vocabulary: every unseen word is unknown and the feature space keeps growing (Heaps' law). BPE fixes the size and handles unseen words with pieces.
- **Does the tokenizer matter for bias?** Pronouns become clean single tokens, so every model can read gender directly. Name pieces can still hint at gender even after pronouns are removed.

Q3 · Representations

"Start thinking how to represent your documents into features. We have experimented with token occurrences and frequencies. Can you think of any other ways?"

The idea. A model needs every bio as a list of numbers of the **same length**. Following the tutorial's "named axes", we use one slot per vocabulary token, so 10,000 slots per bio whatever its length. The three representations differ only in **which number goes into slot *v***:

	Number in slot *v*	In words	Source
R1 indicator	1 if token *v* is in the bio, else 0	is it there?	tutorial

	Number in slot *v*	In words	Source
R2 bag-of-words	how many times *v* occurs	how often?	tutorial
R3 TF-IDF	(1 + ln count) × idf(*v*), then the row is scaled to length 1	how often, how rare, regardless of length	ours

The idf (inverse document frequency) of a token, where N = number of training bios and df(*v*) = how many of them contain *v*:

```
\mathrm{idf}(v) = \ln\frac{1+N}{1+\mathrm{df}(v)} + 1
```

Why TF-IDF as our third representation: it makes three fixes to raw counts.

1. **Repeats count less.** A count of 4 becomes 1 + ln 4 = 2.4: the 4th "patients" tells you less than the 1st.
2. **Common tokens count less.** idf is about 1 for tokens in every bio (*the* 1.25, *he* 1.76, *she* 1.95) and much higher for telling ones (*nurse* 4.45, *attorney* 5.25, *chiroprac* 7.83). With raw counts, *the* would weigh as much as *chiropractic*.
3. **Length is removed.** Each row is divided by its length, so a 116-word bio and a 26-word bio are on the same scale.

Storage. A bio uses a median of 57 different tokens out of 10,000, so 99.4% of the table is zeros. A **sparse (CSR) matrix** stores only the non-zeros: 124 MB instead of 10.3 GB.

Leakage trap. idf is *learned* (it counts bios), so it is fitted on the **training part of each fold only** and then applied to the validation part. In the code, every representation has a `fit` step (training rows) and a `transform` step; for R1 and R2, `fit` does nothing.

What we found

- **78% of non-zero counts are 1** (13% are 2), so R1 and R2 are nearly the same numbers. The tokens that repeat most are punctuation and small words, plus *blue* (from the repeated "Blue Cross Blue Shield" template). Result: **indicator and bag-of-words are practically equal for every model.**
- **TF-IDF helps the log-linear model a lot** (0.749 vs 0.727 indicator, 0.719 counts), but not the MLPs (0.754 vs 0.749 for the MLP: practically equal).

Likely questions

- **What is the dimension of a bio's vector? Does it depend on the bio?** 10,000 (the vocabulary size) for every bio. Length changes only how many slots are non-zero (R1) or how big the counts are (R2); TF-IDF removes even that.

- **Can these tell "man bites dog" from "dog bites man"?** No. All three are bags: word order is thrown away. With a BPE tokenizer the answer is the same; BPE changes the units, not the bag.
- **What would bigrams change?** A slot per *pair* of neighbouring tokens ("law school", "yoga classes") keeps some order and catches phrases. Cost: up to 10,000² possible slots, so it needs a frequency cut-off or hashing.
- **Why does TF-IDF help log-linear but not the MLP?** The MLP's first layer can learn its own weight per token, so it can learn "ignore *the*" by itself. Log-linear with an L2 penalty shrinks all weights equally, so it benefits from inputs that are already sensibly weighted.
- **Why did indicator and counts perform the same?** Bios are short, so 78% of counts are 1: the two representations hold almost the same numbers.
- **Other representations we could have used:** averaged pretrained word embeddings (e.g. fastText: dense, brings outside knowledge, but carries its own gender bias), bigrams, character n-grams, or a pronoun-scrubbed bag-of-words (we tried scrubbing as our extra).
- **How does TF-IDF interact with gender?** It gives pronouns low weight (idf 1.8–2.2), so they influence the vector less. For both neural models, TF-IDF had the smallest gender gaps (RMS 0.149 and 0.142).

Q4 · Architectures

"We had experimented with Logistic Regression and a shallow neural network. You can experiment with different architectures and their hyperparameters (add layers, change the activation functions, dropout rates)."

What all three do. They map a bio's 10,000 numbers to **28 scores** (logits), one per occupation. **Softmax** turns scores into probabilities that sum to 1, and the prediction is the most probable occupation (the tutorial's *most probable class* decision rule).

```
p(k \mid x) = \frac{\exp(s_k(x))}{\sum_{j=1}^{28} \exp(s_j(x))}
```

Training nudges the weights so the correct occupation gets high probability, by minimising the **cross-entropy loss**, −ln p(correct occupation): 0 when the model is sure and right, large when it is sure and wrong.

	A1 log-linear (tutorial)	A2 MLP (tutorial)	A3 deeper MLP (ours)
Structure	scores = W·h(x) + b	10,000 → 128 (ReLU) → 28	10,000 → 128 (LayerNorm, ReLU), + a residual 128 layer → 28
Weights	280,028	1,283,740	1,300,765
Built-in assumption (inductive bias)	clues add up; a word always pushes the same way	learned "detectors" combine words	detectors of detectors, added on top of the first layer
Can model "surgery, unless nurse"?	no	yes	yes
Kept from overfitting by	L2 penalty, strength set by C	dropout 0.1, weight decay, early stopping	dropout 0.3, 20% of tokens hidden, weight decay, early stopping
Training	scikit-learn `LogisticRegression` (lbfgs), convex: one best answer	AdamW, mini-batches of 128	same
Mean macro-F1 (over the 3 representations)	0.732	**0.750**	0.745

Why a deeper MLP as our third? The brief suggests a modification of the MLP, and several occupations share vocabulary (physician / surgeon / nurse; professor / anything academic). We tested whether more combining power separates them. **Not at first**: our first version (a plain 256 → 128 stack, 2.6M weights) was worse than the MLP (mean 0.735; 0.730 vs 0.754 on TF-IDF). Diagnosis: TF-IDF numbers are tiny, so almost no bio-specific signal reached the second layer. The redesign (by a teammate) adds LayerNorm (rescales the hidden numbers, so the input scale no longer matters), a residual shortcut (the second layer's output is added to the first, so it can fall back to the MLP), ReZero (that extra layer starts switched off) and token dropout (20% of each bio's tokens hidden during training). Result: mean 0.745, practically equal to the MLP on indicators and counts, slightly behind on TF-IDF (−0.010, unsure).

How the neural models are trained

- **Mini-batches and epochs:** 128 bios per update step; one pass over all training bios is an epoch (at most 20).
- **AdamW:** the update rule. The **learning rate** is the step size; *weight decay* keeps weights small.
- **Early stopping on an inner hold-out:** 10% of the *training* part is held out. After every epoch we score macro-F1 on it, keep the best epoch, and stop after 3 epochs without improvement. The models overfit fast: the hold-out score peaked after **1 epoch (MLP)** and **about 3 epochs (deeper MLP)**, while training loss kept falling.
- **Why not use the validation fold for this, like the tutorial?** The tutorial's `train_neural_network_model` picks the best epoch on the validation fold and then reports that same fold, which is optimistically biased (like a student choosing which exam questions get graded).
- **Why stop on macro-F1, not the loss?** Macro-F1 is our metric; the loss is dominated by the big classes. In a pilot, loss-based stopping scored about 0.02 lower.
- **Sparse first layer:** `torch.sparse.mm` touches only the ~57 non-zero slots of a bio, so training on a CPU takes 9–28 s per fold.

Tuning: one setting per system. Representations differ in scale (a TF-IDF row has length 1, a count row about 9), so one shared setting could favour some systems. We tried a few **candidates** on a 90/10 split of fold 0's training part (never the validation folds):

Model	Setting	Candidates	Chosen
log-linear	C (how much the model may trust single words; small = cautious)	0.3, 1, 3	0.3 for indicator and counts; **3 for TF-IDF**
MLP	learning rate	0.001, 0.003	0.001 for all
deeper MLP	learning rate	0.001, 0.003	0.001 for all (with LayerNorm, input scale no longer matters)

TF-IDF needed a *weaker* penalty because its numbers are small, so the same penalty hits it harder. With C = 0.3 it would have scored 0.691 instead of 0.757 on the hold-out.

Likely questions

- **Why log-softmax + NLLLoss in the tutorial's code?** Together they compute cross-entropy; working in log space avoids overflow and underflow from exponentials (numerical stability).

- **What does ReLU do, and why is it needed?** max(0, z). Without a non-linearity, stacked linear layers collapse into one linear map, so the MLP would be no more powerful than log-linear.
- **Other activation functions?** tanh and sigmoid saturate (gradients vanish for large inputs); GELU is a smooth ReLU. We kept ReLU: simple, fast, standard.
- **What does dropout do? What rate?** During training it randomly switches off a share of hidden units, so the model cannot rely on a few. In a pilot, 0.1, 0.3 and 0.5 made little difference; we used the tutorial's 0.1 for the MLP and 0.3 plus 20% hidden tokens for the deeper MLP.
- **Why does the deeper MLP not beat the MLP?** A bag of words has little structure beyond the word combinations one layer already learns, and both models overfit within 1–3 epochs. The redesign removed the handicap (it now matches the MLP), but there is nothing extra for a second layer to find.
- **What is the MLP's first layer really doing?** Each token has a learned list of 128 numbers (an embedding); for a bio the layer adds up the lists of its tokens (an *embedding bag*).
- **Convex vs non-convex?** Log-linear training has one valley, so the same answer every time. MLP training has many, so results depend slightly on the random start (seed).
- **Why did `loglin_bow` not converge?** Raw counts are badly scaled (values up to 20+), which slows the optimiser; within 500 iterations it did not fully converge on any fold. Indicators and TF-IDF did.
- **Why Xavier initialisation (a tutorial question)?** It scales starting weights by layer size so the signal neither explodes nor vanishes as it passes through layers.
- **How would you see overfitting?** Training loss keeps falling while hold-out loss rises (our training-curve plot).

Q5 · Evaluation

"Evaluate your models! Was the evaluation method adequate for your new experimental setup? Do you want to assess other aspects of performance?"

Short answer: the tutorial's setup (plain accuracy, unstratified folds, best epoch chosen on the validation fold) was not adequate for our data, so we changed four things, and we added a fairness evaluation.

Tutorial	Ours	Why
accuracy	**macro-F1** (+ accuracy and balanced accuracy reported)	84× imbalance: accuracy rewards ignoring rare jobs

Tutorial	Ours	Why
random folds	**stratified** by occupation × gender (56 groups)	every fold gets the same job mix and the same % women per job
best epoch picked on the validation fold	**inner 10% hold-out** from the training part	the validation fold is used once, so scores are not optimistic
accuracy only	+ per-occupation F1, confusion matrix, **recall gap between women and men**	the dataset's real question is *for whom* the model works

The metrics, in plain words (all 0 to 1, higher is better):

- **Precision** for nurse: when the model says "nurse", how often is it right?
- **Recall** for nurse (= **true positive rate, TPR**): of all real nurses, how many did it find?
- **F1** = 2 × precision × recall / (precision + recall): high only when both are high.
- **Macro-F1** = the plain average of the 28 F1 scores. Rapper counts as much as professor. **Our main metric.**
- **Accuracy** = share of all bios classified correctly. Kept to compare with papers (Ravfogel et al. report it).
- **Balanced accuracy** = the plain average of the 28 recalls.

Cross-validation design

- **3 folds × 2 seeds = 6 scores per system.** Cut the bios into 3 equal parts; train on 2, test on the third; repeat so each part is tested once. Then reshuffle with a second seed and repeat. Each training run uses about 171k bios; each test part has about 86k.
- **The same 6 splits for all 9 systems**, so systems can be compared split by split (a *paired* comparison).
- **Only inside train.** dev and test are untouched this week.
- **Why not 5 folds × 3 seeds?** It took about 1.5 hours instead of 25 minutes. Fewer folds still give large training sets. The price is wider uncertainty in the Bayesian comparison: its correction multiplies the variance by $1/n + 1/(k-1)$, which is $1/6 + 1/2 \approx 0.67$ for us against $1/15 + 1/4 \approx 0.32$ for 5 × 3.
- **Why not 1 seed?** Only 3 scores: too few to conclude anything.

Fairness metrics (details in the Extra section)

- **TPR gap** for an occupation = recall(women) − recall(men). Positive: women in that job are recognised more often. Equal TPRs is the fairness criterion *equality of opportunity* (Hardt et al., 2016).

- **RMS gap** = the square root of the mean squared gap over occupations: one number per system (Romanov et al., 2019).
- These are measured on **out-of-fold predictions combined**: each seed's 3 folds together predict every bio once, so each group has its full size (all 88 female rappers, not ~29 per fold). Occupations count only if both genders have at least 10 bios (all 28 pass).

Likely questions

- **Is accuracy a good metric here? How would you use the data to argue it?** No: count the labels. Professor is 30%, so "always professor" gets 0.30 accuracy with 0.016 macro-F1. Our best model has accuracy 0.81 but macro-F1 0.75: the small classes pull macro-F1 down, and accuracy hides that.
- **Why macro-F1 rather than balanced accuracy?** Balanced accuracy only looks at recall. Macro-F1 also punishes a model that over-predicts a class (low precision), e.g. labelling every academic "professor".
- **Why cross-validation and not one split?** One split gives one score, and that score itself is random (it depends on which bios landed in the test part). Several scores show how much results vary, which the Bayesian comparison needs.
- **Are the 6 scores independent?** No. The training parts overlap (each pair of training parts shares half its bios), so scores move together. The Bayesian *correlated* t-test corrects for this.
- **Why stratify?** With classes as small as 0.35% and groups as small as 88, a random split could give folds with very different mixes; stratification makes every fold a fair copy.
- **Why remove duplicates first?** A copy in both the training and the test part lets a model score by memorising.
- **How do you know the metric code is right?** It reproduces the baselines exactly (always-professor: 0.016 / 0.298 / 0.036), and the gap code gives +0.20 on a hand-made example.
- **What else could you measure?** Precision gaps or false-positive gaps between genders (other fairness notions), calibration of probabilities, training time (we report it: 9 s to 74 s per fold).

Q6 · Which system is best?

"Analyse the various models you assessed and try to decide on which is the best."

Short answer: MLP + TF-IDF has the highest score (macro-F1 0.754), but log-linear + TF-IDF (0.749) is *practically equal* to it by our Bayesian analysis. If you value simplicity and being able to read the weights, log-linear + TF-IDF is an equally defensible choice.

Mean macro-F1 over the 6 splits (fold-to-fold standard deviation 0.001–0.004):

Representation	A1 log-linear	A2 MLP	A3 deeper MLP	Row mean
R1 indicator	0.727	0.749	0.744	0.740
R2 bag-of-words	0.719	0.747	0.747	0.738
R3 TF-IDF	0.749	**0.754**	0.745	0.749
Column mean	0.732	0.750	0.745	

The Bayesian comparison, in plain words (Benavoli et al., 2017; the tutorial's `BenavoliCorrelatedT`):

1. For each of the 6 splits, compute difference = score(B) − score(A). Same split for both, so it is a fair head-to-head.
2. From the 6 differences, compute the **posterior**: a curve over possible values of the *true* average difference, showing how believable each is given what we saw. Narrow curve: fairly sure. Wide curve: unsure.
3. Agree beforehand on a **ROPE** (region of practical equivalence): differences too small to matter. Ours: ±0.01 macro-F1.
4. Read off the area of the curve left of the ROPE = P(A better), inside = P(practically equal), right = P(B better). We call a result clear when one area is at least 0.95.

Why "correlated"? The 6 scores share training data, so they are not independent pieces of evidence. The method inflates the uncertainty to compensate (the Nadeau–Bengio correction, using correlation ρ = 1/k for k folds).

Why a ROPE of 0.01?

- **Practical:** +0.01 macro-F1 means every occupation's F1 rising by a point, or one occupation's rising by 0.28. Smaller than that, we would not switch systems.
- **Above the noise:** about 3× the typical fold-to-fold standard deviation (median 0.0031).
- **Relative to the tutorial:** it uses 0.005 for accuracy; macro-F1 swings more (small classes), so a slightly wider band is reasonable.
- **Checked:** conclusions were recomputed with ROPE 0.005 and 0.02 (sensitivity check).

What the 18 comparisons say (each changes one thing: model with representation fixed, or representation with model fixed)

Question	Comparison	Mean difference	Verdict at ROPE 0.01
Does a hidden layer help?	log-linear → MLP, indicator	+0.021	MLP better (P = 0.996)
	log-linear → MLP, counts	+0.028	MLP better (P = 1.00)
	log-linear → MLP, TF-IDF	+0.005	**practically equal** (P = 0.998)
Does a second layer help?	MLP → deeper, indicator / counts	−0.005 / 0.000	practically equal (P = 0.97 / 0.98)
	MLP → deeper, TF-IDF	−0.010	unsure (P(MLP better) = 0.45, P(equal) = 0.55)
	log-linear → deeper, indicator / counts / TF-IDF	+0.017 / +0.027 / −0.004	deeper better on indicator (P = 0.95) and counts (P = 1.00); equal on TF-IDF
Counts vs presence?	indicator → counts, all 3 models	−0.008 to +0.003	practically equal (P = 0.98–0.99)
Does TF-IDF help?	log-linear: indicator / counts → TF-IDF	+0.021 / +0.030	TF-IDF better (P = 1.00)
	MLP: indicator / counts → TF-IDF	+0.005 / +0.007	practically equal
	deeper MLP	0.000 / −0.002	practically equal (P = 1.00 / 0.99)

Sensitivity: TF-IDF beats counts for log-linear, and both the MLP and the deeper MLP beat log-linear on counts at *every* ROPE (0.005, 0.01, 0.02). With 0.005, several "equal" verdicts become "unsure" or small wins; with 0.02 most become "equal". So differences below 0.01 are real but minor.

Where the best system goes wrong. Easiest: dentist (F1 0.94), professor, attorney (0.87). Hardest: teacher (0.57), pastor (0.61), architect, interior designer, paralegal (0.67). Main confusions: paralegal → attorney (34% of paralegals), chiropractor → physician (24%), surgeon → physician (18%), yoga teacher → teacher (17%), psychologist → professor (15%). These are jobs that share vocabulary, as predicted from the keyword analysis.

Likely questions

- **Why Bayesian estimation instead of a significance test (p-value)?** A test answers "is the difference exactly zero?" and hides how big it is. Estimation gives a full picture of how big the difference plausibly is and how uncertain we are, and can conclude "practically equal", which a test cannot.
- **What is a posterior?** Our belief about the unknown (the true mean difference) *after* seeing the data: prior belief updated by the 6 measured differences.
- **What is the HDI?** The narrowest range of values holding 95% of the posterior: the 95% most believable values.
- **What does P(equal) = 0.998 mean for log-linear vs MLP on TF-IDF?** We are 99.8% sure the true difference lies within ±0.01, so in practice they are equally good.
- **What is the effect size?** Posterior mean difference divided by its standard deviation: how many "units of uncertainty" apart the systems are.
- **What are the model's assumptions?** Differences are normally distributed around an unknown mean; Student-t prior on the mean, uniform prior on the spread; the correlation between folds is fixed at 1/k rather than learned.
- **Why not compare all 36 pairs?** Pairs that change one thing at a time answer clear questions (does layer X help? does representation Y help?); pairs that change two things at once do not tell you which change mattered.
- **How would you pick a final model to deploy?** Retrain the chosen system on all of train, or ensemble the 6 trained copies (e.g. majority vote), then evaluate once on the untouched test set.
- **Does a better score mean the model is fairer?** No. Gender gaps barely changed across systems (RMS 0.142–0.168): a separate question, answered in the next section.

Extra · Fairness: does it work equally well for women and men?

Short answer: no. In every system, the recall gap between women and men follows the share of women in the occupation (correlation 0.83–0.90; 0.86 for the best system). The model **amplifies** the imbalance it learned from, as De-Arteaga et al. (2019) found.

Biggest gaps for the best system (MLP + TF-IDF)

Occupation	% women	Recall women	Recall men	Gap	Missed bios of the less-recognised gender mostly go to
model	83	0.84	0.35	+0.49	teacher (male models)
dietitian	93	0.82	0.51	+0.32	professor (male dietitians)
paralegal	85	0.58	0.34	+0.24	attorney (77% of male paralegals' misses)
nurse	91	0.82	0.66	+0.16	physician (male nurses)

Occupation	% women	Recall women	Recall men	Gap	Missed bios of the less-recognised gender mostly go to
interior designer	81	0.62	0.48	+0.14	architect (male interior designers)
pastor	24	0.42	0.58	−0.15	teacher (female pastors)
surgeon	15	0.48	0.68	−0.21	physician (female surgeons)
rapper	10	0.51	0.72	−0.21	model (female rappers)

The pattern: the minority gender gets pushed to a **related, gender-typical** job. These are the same confusions De-Arteaga et al. report. In a recruiting search for "surgeon", this model would find 68% of male surgeons but only 48% of female surgeons, making a male-dominated group look even more male (**compounding imbalance**).

Across systems: RMS gap 0.142–0.168. Our deeper MLP has the smallest gaps (0.142 on TF-IDF), then the MLP (0.149–0.160), then log-linear (0.159–0.168); for both neural models TF-IDF gives the smallest gap. These differences are small and we did not test them with the Bayesian method.

Deleting pronouns ("scrubbing"). We deleted he, she, him, her, his, hers, himself, herself, mr, mrs, ms from every bio, then retrained the best system on the same 6 splits with the same settings, so the only change was the missing words.

	Original	Pronouns deleted
Bios with a clear pronoun majority	94.9%	0.0%
Macro-F1	0.754	0.751 (practically equal, P = 1.00)
RMS gap	0.149	0.116 (−22%)
Correlation of gap with % women	0.86	0.78

Gaps shrank in 23 of 28 occupations but did not disappear (model still +0.43). Gender still leaks through **first names**, family words (*husband, mother*) and gender-typical topics.

Does the model use *she* directly? Yes. In a log-linear model, weight(*she*) − weight(*he*) per occupation follows the share of women with **r = 0.96** (dietitian, nurse, paralegal +1.3 to +1.6; surgeon about −1.1). But the top words per occupation are content words (*nursing, patients* for nurse; *surgery* for surgeon), so pronouns are a steady secondary nudge that tips ambiguous bios.

Likely questions

- **What is the TPR gap and why that fairness notion?** Recall for women minus recall for men in one occupation. It matches *equality of opportunity*: if you are a nurse, your chance of being recognised as one should not depend on your gender.
- **Why the RMS and not the average gap?** Positive and negative gaps would cancel in an average; squaring makes both count and weights big gaps more.
- **Why is gender learnable if it is not an input?** The text contains it: pronouns alone recover the label in 95% of bios.
- **Why does scrubbing not fix it?** Names, family words and topics are *proxies* for gender. Removing explicit words removes only the most direct cue.
- **Did scrubbing cost accuracy?** No: −0.003 macro-F1, practically equal. The job information is in the content words.
- **Why are small groups a problem?** Recall on 88 female rappers is a noisy estimate; that is why we combine all folds' predictions and require at least 10 bios per group.
- **Is a gap always bias in the model?** Partly it reflects the data (who writes which bio, label noise); the model learns and amplifies it. The harm is in how such a system would be used, e.g. recruiting.

Limitations and what we would do next

If the TA asks "what are the weaknesses of your setup?", these are the honest answers, each with a fix.

Limitation	Why it matters	What we would do
Gender is binary and inferred from pronouns	misses non-binary people; can be wrong	report it as a proxy; never present results as about gender identity
Only 6 CV scores per system (3 folds × 2 seeds)	wider posteriors, more "unsure" verdicts	more seeds when time allows
One hyperparameter tuned per system, on one fold	another setting (dropout, hidden size; the deeper MLP's tricks were checked on one split only) could change the ranking	a small grid search per system
`loglin_bow` did not fully converge	its score may be slightly too low	more iterations, or scale the counts

Limitation	Why it matters	What we would do
Near-duplicate templates can sit in both training and test parts	slightly optimistic scores	group template families into the same fold
Tokenizer trained on texts later used for validation	tiny, unsupervised, same for all systems	train it on `dev` only
Scrubbing leaves first names and proxies	gaps only shrink by 22%	remove names too; try debiasing methods (e.g. Ravfogel et al.'s INLP) in later projects
Bag-of-words ignores word order	phrases like "law school" are lost	bigrams, or sequence models in later projects
Gender-gap differences between systems were not tested with the Bayesian method	small RMS differences may be noise	per-fold gap estimates, then the same correlated t-test

Glossary

Term	Meaning
label / class	what we predict: one of 28 occupations
feature	one number describing the input, e.g. "does the token *nurse* occur?"
token	a unit of text the model sees (a word or a piece of one)
BPE (byte-pair encoding)	learns tokens by repeatedly merging the most frequent pair of neighbouring units
pre-tokenizer	the first rough split (on spaces and punctuation) before BPE merges inside each piece
vocabulary size	how many tokens BPE learns (ours: 10,000)
`[UNK]`	the placeholder for a character never seen in training
indicator (R1)	1 if a token occurs in the bio, else 0

Term	Meaning
bag-of-words (R2)	how many times each token occurs; word order is lost
TF-IDF (R3)	damped count × rarity weight, each bio scaled to length 1
idf	inverse document frequency: high for rare tokens, about 1 for tokens in every bio
sparse matrix (CSR)	stores only the non-zero entries; essential when 99.4% are zeros
logits / scores	the 28 raw numbers a model outputs before softmax
softmax	turns scores into probabilities that sum to 1
decision rule	how probabilities become one answer; ours picks the most probable class (argmax)
cross-entropy loss	−ln p(correct class): what training minimises
log-linear model	scores are a weighted sum of the features (softmax regression)
MLP	a neural network with hidden layers between input and output
hidden layer	in-between numbers the model computes; act as learned feature detectors
ReLU	max(0, z): the non-linearity that lets an MLP model combinations
embedding	a learned list of numbers per token (the MLP's first-layer weights)
inductive bias	the built-in assumption of a model about what patterns look like
convex	a training problem with one best solution
regularisation	anything that stops a model memorising (L2 penalty, dropout, early stopping)
C	inverse strength of log-linear's L2 penalty; small C = cautious model
dropout	randomly switching off hidden units during training

Term	Meaning
weight decay	a small pull of all weights towards 0 (in AdamW)
learning rate	the step size of each weight update
epoch	one pass over all training data
early stopping	stop training when a hold-out score stops improving; keep the best epoch
overfitting	memorising the training data instead of learning patterns that generalise
hyperparameter	a setting chosen before training (C, learning rate), not learned
hold-out	data set aside from training to check or tune a model
cross-validation (CV)	split into k parts, train on k−1, test on the rest, k times
fold	one of the k parts, or one train/test round
seed	the starting value of a random generator; a different seed gives a different but repeatable shuffle
stratification	splitting so each part keeps the same class (and gender) proportions
leakage	evaluation data influencing training, which makes scores too optimistic
label noise	wrong or ambiguous training labels
baseline	a trivial predictor that sets the floor (e.g. always "professor")
precision / recall	of predicted *y*, how many are right / of real *y*, how many were found
TPR (true positive rate)	recall, here measured within one gender group
F1, macro-F1	combines precision and recall / the plain average of F1 over classes
balanced accuracy	the plain average of recall over classes
confusion matrix	rows = true class, columns = predicted class: where the mistakes go

Term	Meaning
out-of-fold predictions	each bio's prediction from the fold in which it was in the test part
TPR gap	recall(women) − recall(men) in one occupation
RMS gap	√(mean of squared gaps): one fairness number per system
equality of opportunity	fairness criterion: equal recall for every group
proxy	something correlated with gender that is not gender itself (names, "husband")
scrubbing	deleting explicit gender words from the text
posterior	belief about an unknown after seeing the data
prior	belief before seeing the data
ROPE	region of practical equivalence: differences too small to matter (ours ±0.01)
HDI	highest density interval: the narrowest range holding 95% of the posterior
effect size	mean difference divided by its standard deviation
correlated t-test	Bayesian comparison of CV scores that accounts for folds sharing training data
Nadeau–Bengio correction	the factor that inflates the variance of CV differences for that sharing
NHST	null hypothesis significance testing (p-values), which the tutorial argues against
LayerNorm	rescales a layer's numbers for each bio to mean 0 and spread 1, so the input's scale (TF-IDF vs counts) stops mattering
residual shortcut	a layer's output is added to its input ($z_2 = z_1 + f(z_1)$), so the layer only learns a correction and can be ignored
ReZero	multiplies that correction by a learned number that starts at 0, so training starts from exactly the smaller model

Term	Meaning
token dropout	during training, hide a random share (ours: 20%) of each bio's tokens, so the model can't lean on single words

Sources

All numbers come from our notebook `notebooks/bias-in-bios-week1.ipynb` (final full run of 6 Nov 2026). Papers cited:

- De-Arteaga et al. (2019), *Bias in Bios: A Case Study of Semantic Representation Bias in a High-Stakes Setting*, FAT* 2019. The dataset, scrubbing, gap vs % women.
- Ravfogel et al. (2020), *Null It Out: Guarding Protected Attributes by Iterative Nullspace Projection*, ACL 2020. The Hugging Face version we use.
- Romanov et al. (2019), *What's in a Name? Reducing Bias in Bios without Access to Protected Attributes*, NAACL 2019. The RMS gap.
- Hardt, Price & Srebro (2016), *Equality of Opportunity in Supervised Learning*, NeurIPS 2016.
- Sennrich, Haddow & Birch (2016), *Neural Machine Translation of Rare Words with Subword Units*, ACL 2016. BPE.
- Benavoli, Corani, Demšar & Zaffalon (2017), *Time for a Change: a Tutorial for Comparing Multiple Classifiers Through Bayesian Analysis*, JMLR 18.
- Corani & Benavoli (2015), *A Bayesian approach for comparing cross-validated algorithms on multiple data sets*, Machine Learning 100. The correlated t-test.
- Nadeau & Bengio (2003), *Inference for the Generalization Error*, Machine Learning 52. The variance correction.
- NLP1 Tutorial 1 notebook and its `part1` code (UvA, 2026).

AI assistance (Claude Code) is logged task by task in `docs/ai_assistance_log.md`.